DocMind Revision

Everything You've Learnt — Notes 1 to 3

Question & Answer Study Guide

Day 1	Day 2	Day 3
System Design	**Docker**	**AWS · CI/CD**
Scaling · Caching · Monitoring	Containers · Images · Compose	ECR · IAM · GitHub Actions

Section 1 — System Design & Scaling

The concepts behind building production-ready systems that handle real load

Q1 What is scalability? Why does it matter for DocMind?

Scalability is a system's ability to handle more work without falling apart. For DocMind, it means the system should keep responding correctly and quickly whether 1 user or 2000 users are sending queries simultaneously. Without it, the first time 15 people use DocMind at once, the server slows to a crawl and users get errors or timeouts.

Q2 What is the difference between vertical and horizontal scaling?

Vertical scaling means making your server bigger — more CPU, more RAM. Like upgrading a bicycle to a motorbike. It works up to a point, then hits a ceiling, and if it crashes, everything goes down.

Horizontal scaling means adding more servers running the same code. Like hiring 5 more chai stall owners instead of buying a bigger stove. This is how every serious production system scales. You can add (or remove) instances based on demand, and if one crashes, others keep running. This is what AWS ECS enables for DocMind.

Q3 What is stateless design and why is it required for horizontal scaling?

A stateless service doesn't remember anything between requests. Every request carries everything the server needs to process it. This is required for horizontal scaling because if User A's session data is stored in Server 1's memory, and their next request goes to Server 2 (which the load balancer might do), Server 2 has no idea who they are and the system breaks. With stateless design, any of your 3 ECS containers can handle any request. State lives in shared external storage: documents in S3, embeddings in Pinecone, sessions in Redis.

- **DocMind rule:** no user data in container memory. Uploads go to S3. Embeddings go to Pinecone. Cache goes to Redis (shared).

Q4 What is a load balancer and what does it do in DocMind's architecture?

A load balancer sits in front of your servers and distributes incoming requests across them. Like a hospital receptionist who looks at which doctor is least busy and sends each patient there — no single doctor gets overwhelmed while others sit idle. In DocMind, the AWS Application Load Balancer (ALB) receives all incoming traffic and routes each request to

one of the ECS container instances (s1, s2, or s3). It also performs health checks — if one container stops responding, the ALB stops sending it traffic automatically.

Q5 What is async code and why is it the most impactful single change for DocMind?

Synchronous (blocking) code does one thing at a time. While waiting for OpenAI to respond (2 seconds), the server does nothing else. User 2 queues behind User 1, User 3 queues behind User 2. With 15 users, the 15th waits 30 seconds.

Async code fires the request and immediately starts handling the next user. While User 1's OpenAI call is in flight, the server handles Users 2, 3, 4 simultaneously. One server handles 15+ concurrent users instead of 1.

For DocMind, this means adding async def to every FastAPI route and await to every call that waits on an external service (OpenAI, Pinecone, S3). This is the highest-impact single change because DocMind's main bottleneck is network I/O — time spent waiting for external API responses.

Q6 What is caching and how does it reduce both cost and latency?

A cache stores the result of an expensive operation so you can return it instantly next time without redoing the work. For DocMind: if 100 users ask the same question about the same document, you should call the LLM once and return the cached answer to the other 99.

Cost reduction: at 2000 DAUs with 10 queries/day, a 30% cache hit rate saves roughly ₹8,900/month in LLM costs. Latency reduction: a cache hit returns in milliseconds; an LLM call takes 1–3 seconds.

- **Cache Aside pattern:** check cache first → if hit, return instantly → if miss, call LLM, store result, return it.
- **Semantic caching:** matches similar questions by meaning, not just exact string — "contract terms" and "terms of the contract" both get the same cached answer. Significant additional cost saving.

Q7 What is a message queue and when would DocMind use one?

A message queue is a buffer for slow background tasks. When a user uploads a PDF, processing it (parsing, chunking, embedding) takes 10–30 seconds. Instead of making the user wait, you accept the upload instantly, drop the job into a queue, and return "processing". A worker picks up the job and processes it in the background. The user gets notified when it's ready.

- This prevents your API from blocking during expensive ingestion operations and allows you to handle upload spikes without crashing.

- **Tools:** AWS SQS (managed, reliable), Celery with Redis (Python-native). DocMind would add this when handling many concurrent uploads becomes a bottleneck.

Q8 What are the three bottleneck types in a system? Which one hits DocMind hardest?

CPU bottleneck: the server runs out of processing power. Fix: add more servers.

Network I/O bottleneck: the server spends most of its time waiting idle for external responses (APIs, databases). Fix: async code and caching.

Storage bottleneck: the database can't handle concurrent reads/writes. Fix: connection pooling, read replicas.

DocMind's primary bottleneck is Network I/O. Every query waits on Pinecone (retrieval) and OpenAI/Bedrock (generation). Async code directly addresses this without adding any infrastructure.

Q9 How would you estimate what DocMind costs at 2000 users?

Start from first principles: 2000 users × 10 queries/day = 20,000 LLM calls/day = 600,000/month.

At GPT-4o mini pricing (~₹0.05 per 1000 tokens, ~2000 tokens per query): ₹600,000 × 0.0001 = approximately ₹60/day = ₹1,800/month.

With a 30% cache hit rate: ₹1,260/month, saving ₹540/month. The cache pays for itself within days.

I track the exact per-query cost in Langfuse, so these aren't estimates — they're measurements I can show on a dashboard.

Q10 What is rate limiting and why does DocMind need it?

Rate limiting caps how many requests a user or IP address can make within a time window. Without it, a single user (or a bug in their code) could send thousands of requests per minute, incurring huge LLM costs and potentially crashing your servers. API Gateway on AWS can enforce this automatically — a common limit for a RAG system would be 20–30 queries per minute per user. Requests over the limit receive a 429 Too Many Requests response.

Q11 What is observability? How is it different from just logging?

Logging is recording what happened: text records of events. "User queried doc X at 14:32."

Metrics are numerical measurements over time: average response time, error rate per minute.

Tracing follows one specific request through your entire system — seeing exactly how long each step took.

Observability is having all three, structured so you can answer any question about your system's behaviour without deploying new code. For DocMind: Langfuse handles tracing (per-query, per-step latency, cost, chunks retrieved). CloudWatch handles metrics (infrastructure health, error rates). Together they give full-stack observability.

Q12 What does Langfuse tell you that CloudWatch doesn't?

CloudWatch watches your infrastructure: is the container healthy, how much CPU is it using, are requests timing out. It tells you that something is wrong. Langfuse watches your AI pipeline: for a specific query, which chunks were retrieved, what prompt was sent to the LLM, what it responded, how many tokens were used, what it cost. It tells you why something is wrong — for example, whether a bad answer was caused by poor retrieval, a bad prompt, or a hallucination.

Q13 What is RAGAS and what does each metric mean?

RAGAS is an evaluation framework for RAG systems. It runs your system against a test set and produces four scores:

- **Faithfulness (0–1):** are all claims in the answer grounded in the retrieved chunks? A score of 0.73 means 27% of responses include information not in the source document — hallucination.
- **Answer Relevancy (0–1):** does the answer actually address the user's question?
- **Context Precision (0–1):** of the chunks retrieved, how many were actually useful for the answer? Low score = your retrieval is noisy.
- **Context Recall (0–1):** did you retrieve all the chunks needed to fully answer the question? Low score = your retrieval is missing important context.

I run RAGAS on a 15-question test set weekly to track quality over time and catch any regressions.

Section 2 — Docker

Containers, images, Dockerfiles, and how DocMind is packaged

Q14 What problem does Docker solve? Why is it critical for deploying DocMind?

Docker solves the "works on my machine" problem. Your code depends on an entire environment — Python version, library versions, OS settings. A different machine has different versions of everything. Docker packages your code and its complete environment into a sealed container. Wherever that container runs — your laptop, a colleague's machine, AWS — it behaves identically. For DocMind, this means you can build the image once on your laptop and deploy it to AWS ECS knowing it will run exactly the same way.

Q15 What is the difference between a Dockerfile, an image, and a container?

Dockerfile: a plain text recipe — step-by-step instructions for building the application environment. You write this once.

Image: the result of following the recipe. A frozen, immutable snapshot of your app and all its dependencies. Stored in ECR. Can be 200MB–1GB.

Container: a running instance of an image. You can run many containers from one image simultaneously. Each is isolated. This is exactly what ECS does — runs 3 containers from your docmind image.

- **Analogy:** Dockerfile = recipe card. Image = finished cake (frozen). Container = a slice being eaten.

Q16 What is Docker layer caching and why does it matter for build speed?

A Docker image is a stack of layers, each representing one instruction in the Dockerfile. Docker caches these layers independently. If a layer's instruction hasn't changed, Docker reuses the cached layer instead of rebuilding it.

- **The practical impact:** COPY requirements.txt then RUN pip install, then COPY your code. If only your code changes, the pip install layer is already cached. First build: 3 minutes. Every subsequent code-only change: 10 seconds.
- **The mistake:** if you COPY all your code first then pip install, every code change invalidates the pip install cache and forces a full reinstall every time.

Q17 Walk me through each line of DocMind's Dockerfile.

The Dockerfile has six key instructions:

- **FROM python:3.11-slim** — base image. Python 3.11, slim variant (130MB not 900MB). Always use slim for production.
- **WORKDIR /app** — set the working directory inside the container. All commands run from here.
- **COPY requirements.txt .** — copy the dependency list first (layer caching trick).
- **RUN pip install --no-cache-dir -r requirements.txt** — install deps. Cached if requirements.txt unchanged.
- **COPY . .** — copy the actual application code. This layer changes often but that's fine — pip install is already cached.
- **EXPOSE 8000 8501** — documentation that the container uses these ports.
- **ENTRYPOINT ["/docker-entrypoint.sh"]** — the script that starts both uvicorn and streamlit when the container boots.

Q18 Why do you have a docker-entrypoint.sh script? What does it do?

DocMind runs two processes: the FastAPI backend (uvicorn on port 8000) and the Streamlit frontend (port 8501). The entrypoint script starts both when the container boots.

```
uvicorn main:app --host 0.0.0.0 --port 8000 &   # background
sleep 3   # wait for uvicorn to be ready
exec streamlit run frontend/app.py ...   # takes over as PID 1
```

- **Why exec?** Without exec, the shell script becomes PID 1. Docker watches PID 1. With exec, streamlit becomes PID 1, so Docker correctly detects if it crashes.
- **Why sleep 3?** Prevents Streamlit making API calls before uvicorn is ready — a race condition.
- **Why --host 0.0.0.0?** Without this, uvicorn only accepts connections from inside the container. The load balancer and your browser are outside. 0.0.0.0 means accept from anywhere.

Q19 What is .dockerignore and why does it matter for security?

The .dockerignore file tells Docker which files to exclude when copying your project into the image. Without it, COPY . . would include your .env file (with real API keys) inside the image. If that image is ever pushed to a public registry, your keys are exposed. The .dockerignore must always include .env, __pycache__, venv/, .git, and any large data files or model weights.

Q20 What is docker-compose and when do you use it?

Docker-compose lets you define and run a multi-service application with a single command. For DocMind, it starts the FastAPI+Streamlit container and a Redis cache

container together, configures the network between them (so the app can reach Redis at redis://redis:6379), and passes environment variables from your .env file.

- **docker-compose up** — start everything.
- **docker-compose down** — stop and clean up.
- Used for local development. In production, ECS handles running the containers instead.

Q21 What does docker run -p 8000:8000 mean?

The -p flag maps a port on your laptop (host) to a port inside the container. The format is host_port:container_port. So -p 8000:8000 means: when traffic hits port 8000 on my laptop, forward it to port 8000 inside the container. The app inside always listens on 8000 (as per the CMD). If you used -p 9000:8000, you'd visit localhost:9000 in your browser and Docker would forward it internally to container port 8000.

Q22 Why are you running both Streamlit and FastAPI in one container rather than two?

Two containers is the production best practice when the frontend is a separate application with its own runtime — a React app with Node.js, for example. That way the two halves scale independently and deploy independently.

- Streamlit is a Python library, not a separate runtime. It shares the same Python environment, the same dependencies, the same process space as FastAPI. Splitting them into two containers would add complexity (a Docker network to manage, two sets of environment variables, two build processes) without any real benefit.
- **If we moved to React:** I'd split them immediately. Different runtime (Node vs Python), different scaling needs, different deployment lifecycle. That's when the two-container pattern earns its complexity.

Section 3 — AWS & CI/CD

Cloud infrastructure, ECR, IAM, and automated pipelines

Q23 What is AWS and why are we using it for DocMind instead of just running everything locally?

AWS is Amazon's cloud computing platform — a global collection of infrastructure services you rent rather than own. Running DocMind locally means it's unavailable when your laptop is off, unreachable by other users, and can't scale. AWS gives DocMind a permanent home with a public address, 24/7 availability, automatic scaling, and managed services (databases, load balancers, monitoring) you don't have to build yourself.

Q24 What is IAM and why should you never use the AWS root account for daily work?

IAM (Identity and Access Management) is AWS's security system. It controls who can do what in your AWS account. Every action is a permission that must be explicitly granted.

- **Root account:** the master key. Has access to everything including billing, account deletion, and overriding all security policies. If compromised, everything is at risk.
- **IAM user:** a named identity with specific permissions. For daily work, you create an IAM user with only the permissions needed — ECR, ECS, S3, CloudWatch. If this gets compromised, the damage is limited.
- **Principle of least privilege:** every identity gets the minimum permissions needed to do its job. Nothing more.

Q25 What is an AWS Access Key and what is it used for?

An Access Key is a credential pair (Access Key ID + Secret Access Key) that programs use to authenticate with AWS. Unlike a web console login (username + password), access keys are used by the AWS CLI, GitHub Actions, and your application code when they need to interact with AWS services. The Secret Access Key is shown exactly once when created — you must save it immediately. If lost, you create a new key pair and revoke the old one.

Q26 What is ECR and how does it fit into DocMind's deployment pipeline?

ECR (Elastic Container Registry) is AWS's private Docker image registry. It stores your Docker images inside your AWS account, private by default. ECS pulls images from ECR to run containers. The flow is: build image locally → push to ECR → ECS pulls from ECR →

container runs and serves users. Using ECR instead of DockerHub keeps your images private, within the AWS network (faster ECS pulls), and under IAM access control.

Q27 Walk me through the command to authenticate Docker to ECR.

Before Docker can push to ECR, it needs temporary credentials from AWS:

```
aws ecr get-login-password --region ap-south-1 \
 | docker login --username AWS --password-stdin \
 <account-id>.dkr.ecr.ap-south-1.amazonaws.com
```

- **get-login-password:** asks AWS for a temporary password (valid 12 hours).
- **pipe (|):** sends that password directly to docker login.
- **--username AWS:** ECR always uses 'AWS' as the username.
- After this, docker push commands to that ECR registry succeed.

Q28 What is ECS? Explain Cluster, Service, Task Definition, and Fargate.

ECS (Elastic Container Service) is AWS's service for running Docker containers at scale:

- **Task Definition:** the blueprint. Which image to run, how much CPU/RAM, which ports, which environment variables. Like a specification sheet.
- **Task:** one running container instance created from a Task Definition.
- **Service:** keeps N tasks running at all times. If a task crashes, the service starts a replacement automatically. This is what gives you reliability.
- **Cluster:** the logical grouping of compute resources where tasks run.
- **Fargate:** serverless compute for ECS. AWS manages the underlying servers. You define CPU and RAM requirements and pay per second of usage. No EC2 instances to manage, patch, or worry about.

Q29 What is CI/CD? What does each part do?

CI (Continuous Integration) automatically builds and tests your code every time someone pushes to the repository. The goal is to catch problems immediately rather than discovering them days later when multiple changes have accumulated.

CD (Continuous Deployment) automatically deploys code that has passed CI to a live environment. When CI succeeds, CD picks up and pushes the new version to production without manual steps.

For DocMind: CI = every push to main triggers docker build and pushes the image to ECR. CD (Day 9 work) = after the image is in ECR, automatically update the ECS service to run the new image.

Q30 What is GitHub Actions? How does it know when to run?

GitHub Actions is GitHub's built-in automation system. You define workflows in YAML files in the .github/workflows/ folder. Each workflow has a trigger (on:) that defines what event causes it to run.

- For DocMind: on: push: branches: [main] — the workflow runs every time code is pushed to the main branch.
- GitHub spins up a fresh Ubuntu virtual machine (the runner), checks out your code, runs your steps in order, then shuts the VM down. You pay nothing for this — free tier includes 2000 minutes per month.

Q31 How do you keep AWS credentials secure in GitHub Actions?

GitHub Secrets. You store your AWS Access Key ID and Secret Access Key in GitHub's encrypted vault (repository Settings → Secrets and variables → Actions). In your workflow YAML you reference them as ${{ secrets.AWS_ACCESS_KEY_ID }}. GitHub injects the real value at runtime. In workflow logs, secrets are always shown as *** — nobody can see the actual value, not even repository administrators. The key rule: credentials are never written in the workflow YAML file, which is committed to the repository and visible to anyone with access.

Q32 Why do you tag Docker images with the git commit SHA instead of just 'latest'?

The commit SHA (e.g. a3f9c2b) is unique to each commit. Tagging an image with the SHA creates a permanent, traceable link between a running container and the exact version of code inside it. If production breaks, you check which image tag is deployed, look up that SHA in your git history, and see exactly what changed and when. The 'latest' tag is overwritten on every build — you lose the ability to trace which code is running. Both tags are pushed: SHA for traceability, latest for convenience.

Q33 What is a GitHub Actions workflow_dispatch trigger?

workflow_dispatch is a trigger that adds a "Run workflow" button in the GitHub Actions UI. In addition to automatic triggers (like on push), it lets you manually start a workflow run at any time without pushing new code. Useful for redeploying without a code change, triggering a one-off task, or testing the pipeline during development.

Q34 If your GitHub Actions pipeline fails, what are the most common causes and how do you debug them?

The most common failure points:

- **AccessDenied error:** your IAM user is missing a required permission. Go to IAM, find the user, check which policy needs to be added.
- **Secret not found:** a typo in the secret name in GitHub or in the workflow file. Check Settings → Secrets → Actions to verify exact names.
- **Docker build fails:** requirements.txt missing from repo, or a library in requirements.txt doesn't exist. Check the Build step logs.
- **ECR push denied:** Docker authentication step failed or token expired. Usually fixed by checking the Login to ECR step output.

Debugging process: click the failing step in the GitHub Actions log — the exact error message is almost always there.

Section 4 — DocMind Architecture

Putting it all together — the system as a whole

Q35 Walk me through DocMind's complete production architecture.

Starting from a user request and tracing it through the system:

- **API Gateway:** receives the request. Checks authentication, applies rate limiting (max 30 queries/min/user), terminates SSL.
- **Application Load Balancer:** distributes the request to one of the ECS container instances using least-connections routing.
- **ECS container (s1/s2/s3):** the FastAPI async app receives the query. First checks Redis.
- **Redis cache:** shared across all containers. If the same question was asked recently, returns the cached answer instantly. Cache hit → response in milliseconds.
- **Pinecone:** on cache miss, queries the vector database to retrieve the top-k most relevant document chunks.
- **AWS Bedrock / OpenAI:** generates a grounded answer from the retrieved chunks and the user's question.
- **Cache write-back:** the new answer is stored in Redis for future identical queries.
- **Langfuse:** the complete trace (latency per step, tokens used, cost in USD, chunks retrieved) is logged.
- **Response:** sent back to the user.

S3 stores all uploaded PDFs. CloudWatch monitors infrastructure health. GitHub Actions CI/CD keeps the image in ECR up to date on every merge to main.

Q36 How does DocMind scale to 2000 users?

Four layers working together:

- **Async FastAPI:** one container handles 15+ concurrent requests instead of 1, because it doesn't block while waiting for Pinecone and OpenAI responses.
- **Multiple ECS instances:** 2–3 container instances behind the load balancer multiply throughput proportionally.
- **Redis cache:** at 30% cache hit rate, 6000 fewer LLM calls per day. Reduces both cost and the load on Pinecone and the LLM.
- **ECS auto-scaling:** CloudWatch metrics trigger adding new container instances when CPU exceeds a threshold, then scales back down when traffic drops.

Q37 What would you change about DocMind if you had more time?

Three things in priority order:

- **Document-level access control:** currently any user can query any document. In production, each chunk should be tagged with an owner ID, and retrieval should filter by the authenticated user's permissions.
- **Background ingestion queue:** move PDF processing (chunking + embedding) to an async queue (SQS + worker). Currently it blocks the API request, which is fine for small documents but breaks for large ones.
- **LangGraph agent:** evolve DocMind from a reactive RAG system to an agent that can decide whether to retrieve from the vector store, search the web, or ask a clarifying question before answering.

Q38 What is the difference between API Gateway and a Load Balancer?

API Gateway is the front door: it handles authentication (who are you?), rate limiting (how many requests are you allowed?), SSL termination, and routing to different services. It's concerned with the what and who of requests.

The Load Balancer distributes traffic: it takes requests that have already passed the gateway and decides which backend instance handles each one. It's concerned with the where — which of the 3 ECS containers gets this request, based on load.

In DocMind's architecture: user → API Gateway (identity and rules) → ALB (traffic distribution) → ECS containers (actual processing).

Q39 What is MCP (Model Context Protocol) and why does it matter?

MCP is a standard proposed by Anthropic for how AI models communicate with external tools — databases, APIs, file systems, other services. Currently, every AI application builds custom connectors for every tool it needs. MCP proposes a universal plug format: if a tool implements MCP, any AI model that supports MCP can use it without custom integration code. It's early-stage and evolving, but it's the direction the industry is moving for AI agent tool-use standardisation.

Q40 What is an AI agent and how is it different from a RAG system like DocMind?

A RAG system is reactive: user asks a question → retrieve relevant chunks → generate an answer → done. It follows a fixed pipeline.

An agent can think, plan, and take actions. Given a question, it decides: do I need to look this up in the vector store? Search the web? Call an API? Do a calculation first? Ask a clarifying question? It can loop, branch, and chain multiple steps before producing an answer.

DocMind today is a RAG system. The natural next evolution is a LangGraph agent that uses the RAG pipeline as one of several tools it can choose from.

Section 5 — Quick Reference

One-line definitions for every term — flashcard-style

Term	Definition
Horizontal scaling	Add more servers. The right answer for DocMind at scale.
Vertical scaling	Make the server bigger. Has a ceiling. Single point of failure.
Stateless service	Server remembers nothing between requests. Required for horizontal scaling.
Load balancer	Routes incoming requests across multiple server instances.
async / await	Non-blocking code. Server handles other requests while waiting for I/O.
Cache hit	Answer found in cache. Returned instantly at no LLM cost.
Cache miss	Answer not in cache. Must call the full pipeline.
Semantic caching	Cache based on meaning, not exact string. Catches similar questions.
TTL	Time-to-Live. How long a cached answer stays valid before expiring.
Message queue	Buffer for slow background jobs. Accepts work instantly, processes later.
Connection pooling	Reuse pre-opened DB connections. Avoids expensive reconnection per request.
Rate limiting	Cap requests per user per minute. Prevents abuse and cost overruns.
Dockerfile	Text recipe for building a Docker image.
Docker image	Frozen, immutable snapshot of your app and its environment.
Docker container	A running instance of an image. Isolated from other containers.
Layer caching	Docker reuses unchanged layers. Makes rebuilds fast (10s vs 3min).
.dockerignore	Excludes files from the image. .env must always be here.
docker-compose	Starts multiple services together with one command.
ENTRYPOINT	Script or command that runs when a container boots.
AWS	Amazon Web Services. Cloud infrastructure rented per second.
IAM	Identity and Access Management. Who can do what in AWS.
Least privilege	Give every identity the minimum permissions it needs. Nothing more.
Access key	Credential pair (ID + secret) for programs to authenticate with AWS.
ECR	Elastic Container Registry. Private Docker image storage in AWS.
ECS	Elastic Container Service. Runs Docker containers at scale in AWS.
Fargate	Serverless compute for ECS. No servers to manage.

Task Definition	Blueprint for ECS: which image, CPU/RAM, ports, env vars.
ECS Service	Keeps N tasks running. Replaces crashed tasks automatically.
Region	Geographic AWS data centre location. ap-south-1 = Mumbai.
AWS CLI	Command-line tool for controlling AWS from your terminal.
GitHub Actions	GitHub's built-in CI/CD automation system.
Workflow	A YAML file defining automated tasks triggered by events.
Runner	Fresh Ubuntu VM GitHub provides to run a job. Free tier: 2000 min/month.
GitHub Secret	Encrypted value GitHub injects at runtime. Never visible in logs.
github.sha	The git commit hash that triggered the workflow. Used as Docker image tag.
CI (Continuous Integration)	Auto-build and test on every code push.
CD (Continuous Deployment)	Auto-deploy passing builds to production.
Langfuse	LLM observability tool. Traces every query: cost, latency, chunks, tokens.
RAGAS	Evaluation framework for RAG. Scores faithfulness, relevancy, precision, recall.
Faithfulness	RAGAS metric: is the answer grounded in retrieved context? Measures hallucination.
API Gateway	Front door: authentication, rate limiting, SSL before the load balancer.
CloudWatch	AWS monitoring: container health, error rates, cost, infrastructure metrics.
MCP	Model Context Protocol. Standard for AI models to use external tools.
Agent	AI that can plan and take multi-step actions, not just answer one-shot queries.

The End.